CSE 323: Operating System Design

File System

Salman Shamil



North South University (NSU)
Summer 2026

Original slides by Mathias Payer and Sanidhya Kashyap [EPFL]

Lecture Topics

- Filesystem API
- Internal and external interface
- Inodes and devices
- Filesystem implementation

This slide deck covers chapters 39, 40, 41 in OSTEP.

Purpose of a File System (1/2)

- Given: set of persistent blocks from a storage device
- Goal: manage these blocks efficiently.

Purpose of a File System (1/2)

- Given: set of persistent blocks from a storage device
- Goal: manage these blocks efficiently.

- Manages data (mostly) on nonvolatile storage
- Enables users to name and manipulate semi-permanent files
- Provide mechanisms to organize files and their metadata
  - e.g., owner, permissions, or type

Purpose of a File System (2/2)

- Map bytes on disk to "file"
- Share files (concurrently?) among users and processes
  - Decide on locking granularity and binding operations
  - Semantics of operations like truncating in a shared world
- File caching: metadata and contents

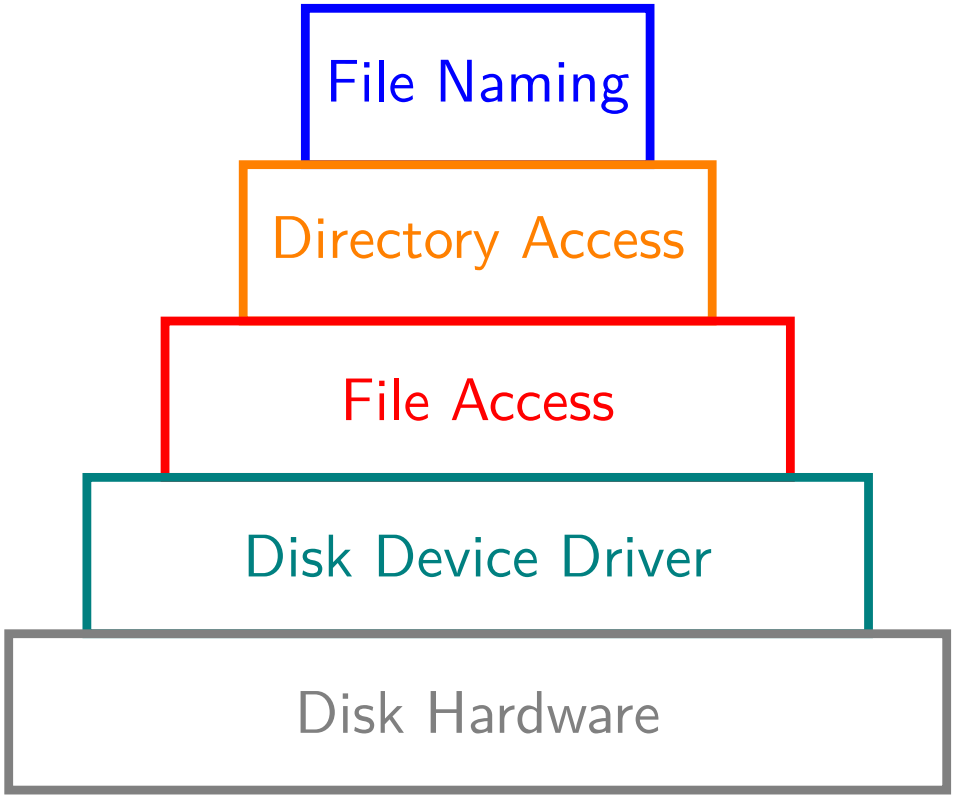
The `file` Abstraction

- A file is a linear persistent array of bytes
  - Data operations: read or write
  - Metadata operations: create, delete, modify permissions/user/...
- Directory contains subdirectories
  - List of directories, files, inode mappings

The `file` Abstraction

- Different perspectives
  - File name (human readable)
  - Inode and device number (persistent ID)
  - File descriptor (process view)

The I/O Hierarchy



- Each level adds functionality (and complexity)

Different Functionalities

- ***Naming***
  - specifies name syntax and encoding
  - e.g., a URL; may be aware if a file is local/remote
- ***Directory access***
  - map name to file object
  - resolve a string to an object
- ***File access***
  - concerns file operations
  - e.g., create/delete/read/write

Different File Philosophies

- Typed files: associate structure
  - System defines *all* possible file types (e.g., text document, source file, html file)
  - File type set at creation, file type specifies operations
- Untyped files: array of bytes
  - File is a sequence of bytes
  - System does neither understand nor care about contents
  - File operations apply to all files

Different File Philosophies

- Typed files: associate structure
  - System defines *all* possible file types (e.g., text document, source file, html file)
  - File type set at creation, file type specifies operations
- Untyped files: array of bytes
  - File is a sequence of bytes
  - System does neither understand nor care about contents
  - File operations apply to all files
- Modern systems settled on untyped files.
- Each file now has attributes or even extended attributes.
  - executable, append only, COW etc.

Desired File Operations

- `create`: create a new file
- `unlink`: remove/destroy a file
- `open`: map a path to a file identifier
- `close`: close a file identifier
- `read`: read from the current file position
- `write`: write to the current file position
- `seek`: modify the file position
- `control`: various control operations such as changing permissions or user
- `createdir`: create a new directory
- `rmdir`: remove a directory
- `readdir`: return all files in a directory

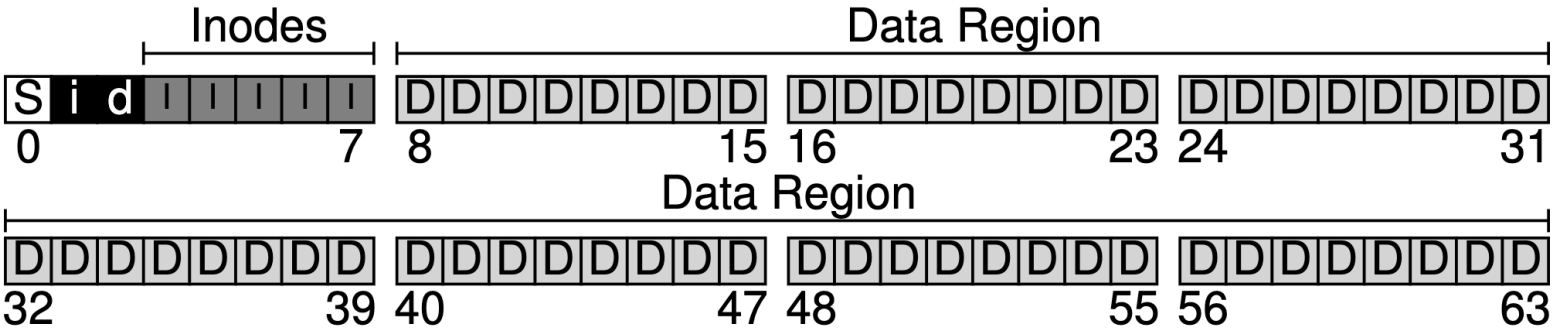
Desired Cost of File Operations

- Sequential read/write is common
  - Design goal: $\mathcal{O}(\texttt{size_of_transfer})$
- Random access (seeking) is infrequent
  - Design goal: $\mathcal{O}(\log(\texttt{file_length}))$

Desired Cost of File Operations

- Sequential read/write is common
  - Design goal: $\mathcal{O}(\texttt{size_of_transfer})$
- Random access (seeking) is infrequent
  - Design goal: $\mathcal{O}(\log(\texttt{file_length}))$
- Constraints and observations
  - Many files are small
  - A few files are large
  - Most accesses are sequential, few accesses are at random positions

Overall Organization



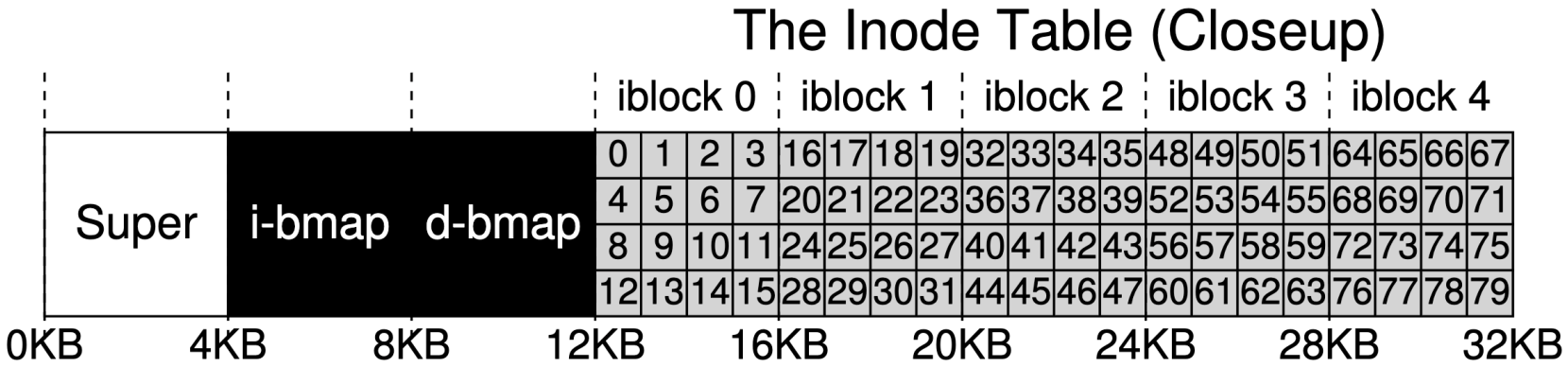
- **Data blocks:** store file contents in one or more blocks
- **Inodes:** store metadata for each file (block locations, permissions, etc.)
- **Inode blocks:** blocks that contain one or more inodes
- **Bitmaps / free lists:** track which inodes and data blocks are free
- **Superblock:** contains the master layout (which blocks hold inodes, data, bitmaps, etc.)

Inode Table

The inode is the generic name that is used in many file systems to describe the structure that holds the metadata for a given file, such as its length, permissions, and the location of its constituent blocks.

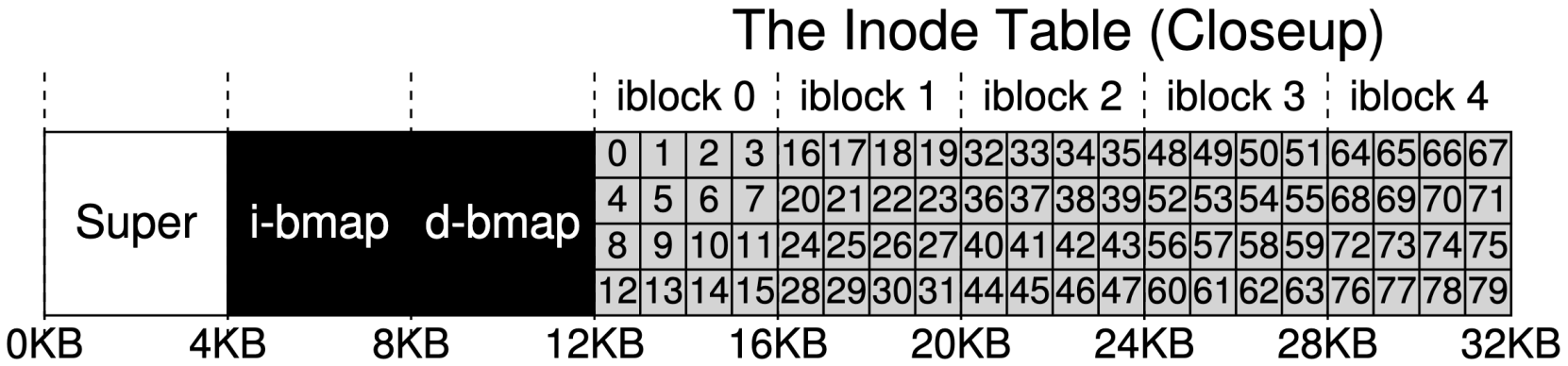
Inode Table

The inode is the generic name that is used in many file systems to describe the structure that holds the metadata for a given file, such as its length, permissions, and the location of its constituent blocks.



Inode Table

The inode is the generic name that is used in many file systems to describe the structure that holds the metadata for a given file, such as its length, permissions, and the location of its constituent blocks.




```
blk = (inumber * sizeof(inode_t)) / blockSize;
sector = ((blk * blockSize) + inodeStartAddr) / sectorSize;
```

Inode Example

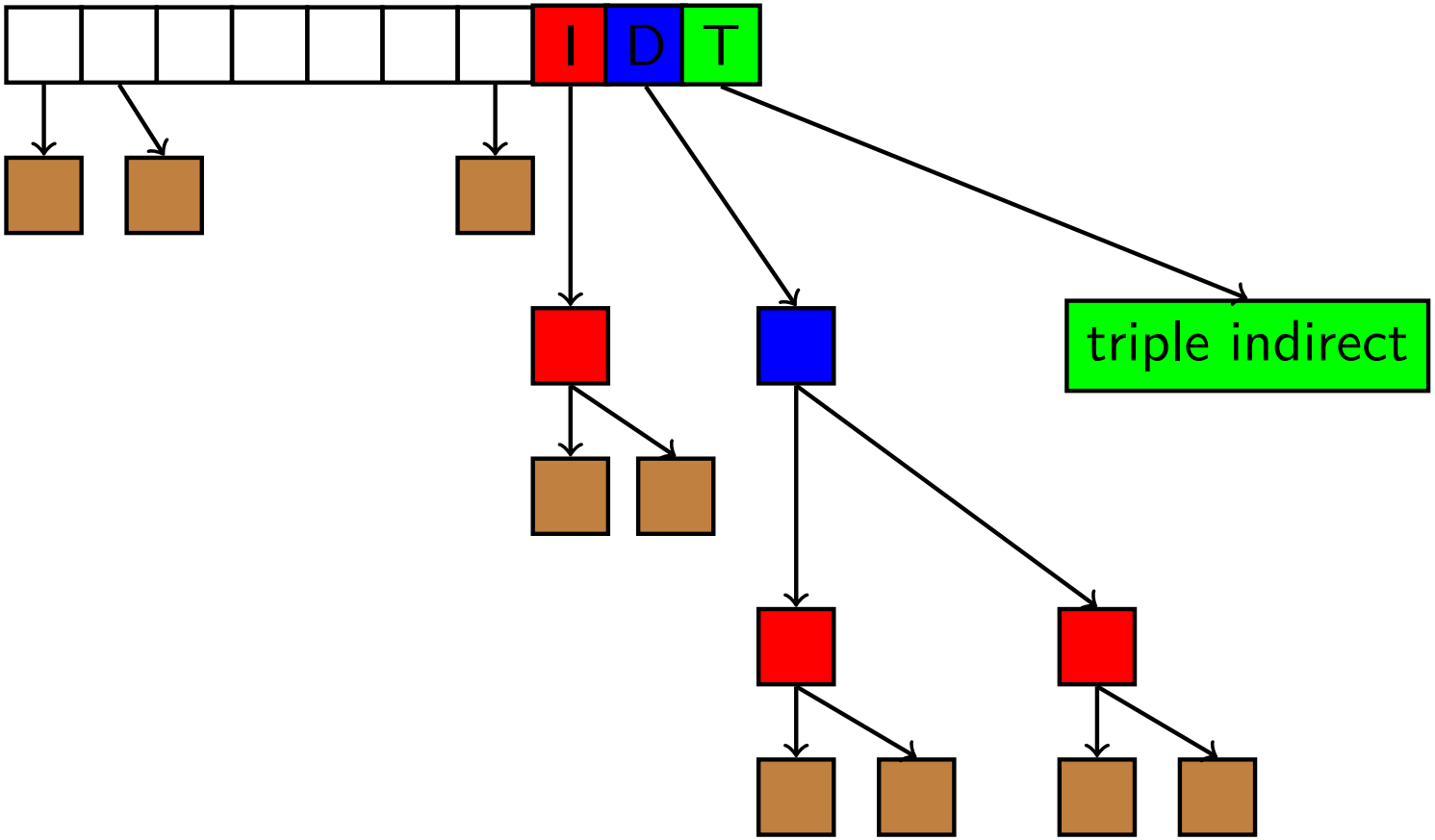
Size	Name	What is this inode field for?
2	mode	can this file be read/written/executed?
2	uid	who owns this file?
4	size	how many bytes are in this file?
4	time	what time was this file last accessed?
4	ctime	what time was this file created?
4	mtime	what time was this file last modified?
4	dtime	what time was this inode deleted?
2	gid	which group does this file belong to?
2	links_count	how many hard links are there to this file?
4	blocks	how many blocks have been allocated to this file?
4	flags	how should ext2 use this inode?
4	osd1	an OS-dependent field
60	block	a set of disk pointers (15 total)
4	generation	file version (used by NFS)
4	file_acl	a new permissions model beyond mode bits
4	dir_acl	called access control lists

Figure 1: Simplified Ext2 Inode

Multi-Level Index (1/3)

- Inodes extended using indirect pointer blocks
- **Single-indirect**: points to a block of data-block pointers
- **Double-indirect**: points to a block of single-indirect pointers
- Enables much larger files than direct pointers alone
- Not a symmetric hierarchical structure like page table
  - Most files are small
  - first few block numbers of a file are made available easily
  - without accessing multiple levels of inode

Multi-Level Index (2/3)



Multi-Level Index (3/3)

- Double-indirect adds ~1,000,000 data blocks $\rightarrow$ ~4 GB (with 4 KB blocks)
- Structure forms an imbalanced pointer tree
- Capacity: (12 direct + 1024 indirect + 1024^2 double-indirect) $\times$ 4 KB $\approx$ 4 GB
- Adding a **triple-indirect** block grows capacity massively ($\approx$ terabytes)

Directory Structure

- Directories map filenames to inode numbers using directory records.
- Records may be stored as fixed-size arrays, linked lists, or more complex structures (hash tables, BSTs, etc.).
- To look up a filename:
  - Fetch inode of directory $\Rightarrow$ locate its data blocks $\Rightarrow$ read data blocks.
  - Search the directory's data (array/list/tree) to find the matching name and retrieve its inode number.

Pathnames

- File identified in filesystem by its **pathname**, starting at root directory.
- When we want to open/read/write a file, we need to find its **inode number**.
- **Given a pathname of a file, how to locate its inode number?**
  1. Start with root directory inode (well known).
  2. For every element (directory) in the pathname:
     - Read directory data blocks.
     - Look up the next element's filename in the directory.
     - Retrieve the inode number of the next element.
  3. Repeat above process recursively until
     - the entire pathname is traversed, and
     - we find the inode number of the desired file in its parent directory.

File Descriptor (1/4)

- The combination of file names and inode/device id are sufficient to implement persistent storage

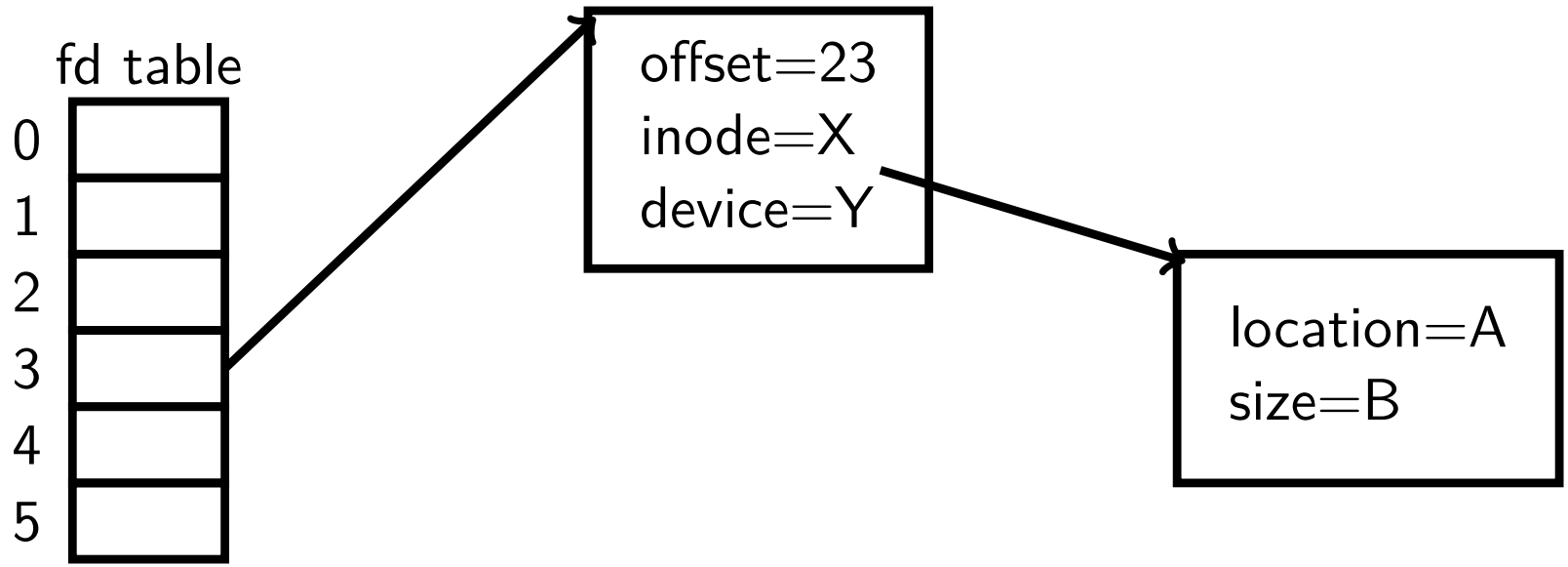
File Descriptor (1/4)

- The combination of file names and inode/device id are sufficient to implement persistent storage
- Drawback: constant lookups from file name to inode/device id are costly

File Descriptor (1/4)

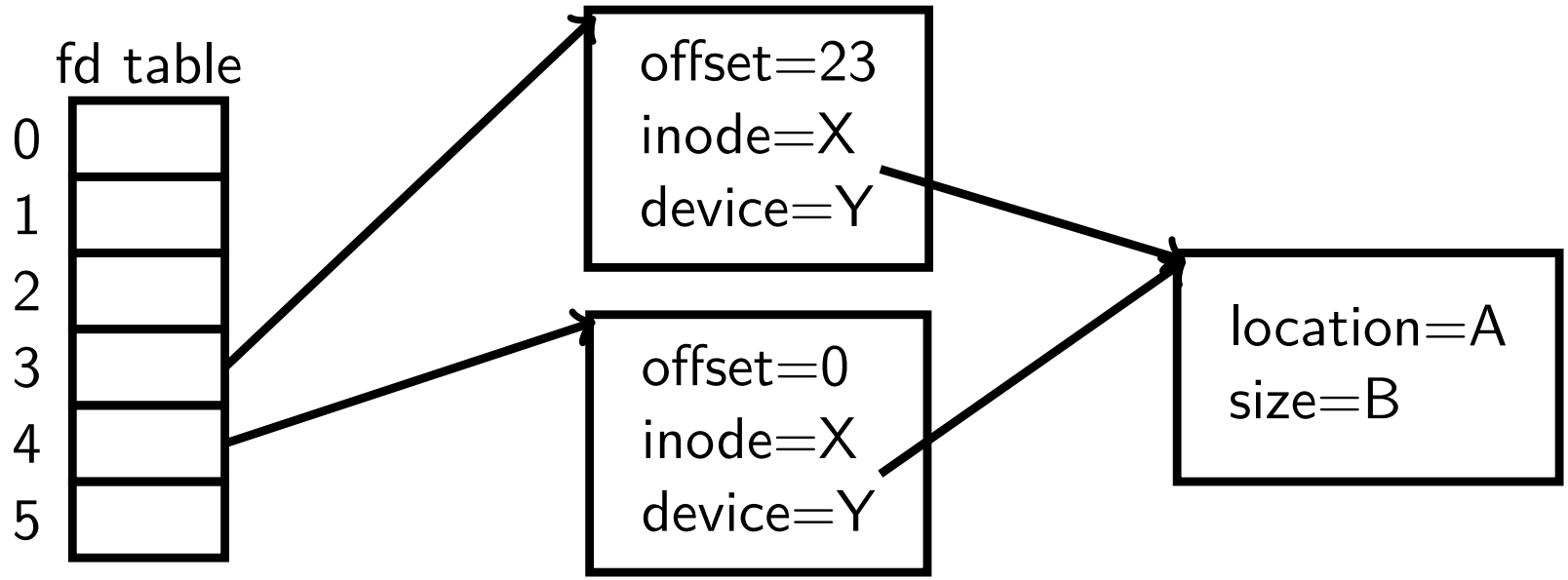
- The combination of file names and inode/device id are sufficient to implement persistent storage
- Drawback: constant lookups from file name to inode/device id are costly
- Idea: do expensive tree traversal once, store final inode/device number in a per-process table
  - Also keep additional information such as file offset
  - Per process table of open files
  - Use linear numbers (fd 0, 1, 2, ...), reuse when freed

File Descriptor (2/4)



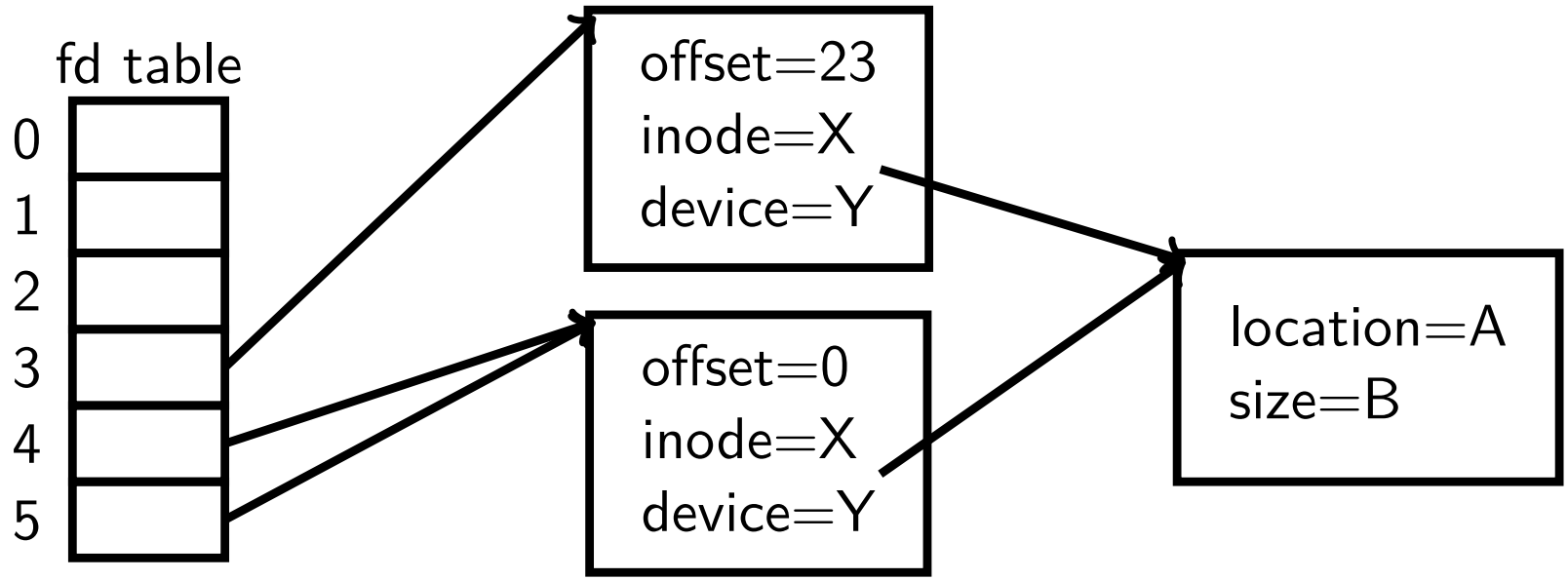

```
int fd1 = open("file.txt");  // returns 3
read(fd1, buf, 23);
```

File Descriptor (3/4)




```
int fd1 = open("file.txt");  // returns 3
read(fd1, buf, 23);
int fd2 = open("file.txt");  // returns 4
```

File Descriptor (4/4)




```
int fd1 = open("file.txt");   // returns 3
read(fd1, buf, 23);
int fd2 = open("file.txt");   // returns 4
int fd3 = dup(fd2);           // returns 5
```

File Deletion

- There is no system call to `delete` files!
- `unlink()` removes a file from a directory and reduces the reference count
- Inodes are marked free if there are no more references to them (that's why they have a reference count)
- File descriptors are freed upon `close()` or when the process exits

Hard Links and Soft Links

- **Links** are pointers to files. They **do not contain the file data** themselves, but reference another file.
- Hard Link
  - A directory entry that **points directly to the inode** of an existing file.
  - Increases the **reference count** of the inode.
  - The inode **does not store the file name**, only data and metadata.
  - The file is only deleted from disk when **all hard links** to it are removed.
- Soft Link (Symbolic Link)
  - A directory entry that **points to a file name**.
  - The OS **resolves the file name** each time the link is accessed.
  - If the original file is deleted, the soft link becomes **dangling**.

Summary

- Filesystem API: handle interaction with the file system
- Internal and external interface
  - Internal: data structures handle large chunk of blocks
  - External: standardized interface
- Three ways to identify a file
  - File names (for humans)
  - Inodes and devices (on the disk)
  - File descriptors (for a process)
- Filesystem implementation
  - Inodes for metadata
  - Bitmaps for inodes/data blocks
  - Superblock for global metadata